PROJECT REPORT

APPLYING ZERO TRUST ARCHITECTURE TO A VOICE-CONTROLLED AI ASSISTANT

Design, Implementation and Evaluation of Guts AI

Submitted by

Abhiraj – A010178725034

Jagrit Chugh – A010178725057

Ira Singh – A010178725045

Vridhi Mangla – A010178725051

Mahek Sood – A010178725015

Kshaunish Sahota – A010178725049

Amity Institute of Information Technology

BSc. IT Big Hons.

Amity University, Noida

October 2026

Abstract

Voice-controlled AI assistants that can invoke tools—such as opening applications, editing files, and sending messages—give a language model practical authority over a personal computer. Conventional perimeter-based security is insufficient because harmful inputs may originate from spoken commands, documents read by the assistant, or model-generated actions. This report applies the Zero Trust Architecture (ZTA) described in NIST SP 800-207 to Guts AI, a Python-based Hinglish voice assistant for Windows that uses a streaming live model and declares 51 tools. The design treats the language model as an untrusted component and implements seven controls: local face verification, spoken confirmation evaluated outside the model, an allow-listed tool set without arbitrary shell access, path and file policies with scoped code execution, separated API keys, recoverable actions with content treated as data, and event logging. In a test of 26 consecutive face-verification readings, all readings exceeded the owner-match threshold of 0.32; the mean score was 0.644, with a minimum of 0.47 and maximum of 0.76. A code scanner also blocked an unreviewed generated program containing eval(). The evaluation is limited to one enrolled user and one session, and several threat scenarios are design analyses rather than adversarial tests. The report concludes that security enforcement should reside in independent code rather than depend solely on model behavior.

Table of Contents

1. Introduction

2. Zero Trust Architecture

3. Threat Model

4. System Overview of Guts AI

5. Zero Trust Design

6. Implementation Details

7. Evaluation and Results

8. Discussion and Limitations

9. Conclusion

Acknowledgment

References

1. Introduction

For many years, enterprise security relied on the assumption that systems inside a network boundary could be trusted. This assumption has weakened as users work from different locations and applications operate across multiple environments. Zero Trust, popularized by Kindervag and implemented at scale through approaches such as Google BeyondCorp, shifts security decisions away from network location and toward users, devices, and individual resources.

The same challenge appears in tool-using AI assistants. A natural-language instruction is converted into a function call that may modify files, execute code, or interact with communication accounts. Prompt injection, excessive agency, speech-recognition errors, and model mistakes can therefore have direct consequences. The assistant requires an access-control layer that remains effective even when the model is mistaken or manipulated.

This report studies that security layer through Guts AI, a working Windows voice assistant written in Python. It listens in Hinglish, responds through a streaming live model, and can invoke 51 declared tools. The report maps the three central Zero Trust principles—verify explicitly, use least privilege, and assume breach—to seven controls implemented around the assistant.

1.1 Objectives

- Develop a threat model for a voice-controlled assistant with computer access.
- Map Zero Trust principles to practical, enforceable controls.
- Describe the architecture and implementation of Guts AI.
- Present the available face-verification test and scenario walkthroughs.
- Identify limitations and future improvements.

1.2 Contributions

The work contributes a threat model, a practical mapping of Zero Trust principles to seven controls, an implementation in which face verification and spoken confirmation are evaluated outside the language model, and an initial evaluation based on real face-verification readings and scenario analysis.

2. Zero Trust Architecture

2.1 Core Principles

Zero Trust is a security strategy rather than a single product. NIST SP 800-207 describes an approach in which no user, device, or network location receives implicit trust. Each access request is evaluated and access is restricted to the resource required.

Verify explicitly: Evaluate requests using relevant signals such as identity, device condition, and resource sensitivity rather than relying on network position.

Least privilege: Provide only the minimum permissions needed for a task and limit their duration.

Assume breach: Design as though an attacker may already be present, so that compromise of one component does not automatically expose all resources.

2.2 Logical Components

The NIST model includes a Policy Engine (PE), which decides whether access should be granted; a Policy Administrator (PA), which establishes or terminates the communication path based on that decision; and a Policy Enforcement Point (PEP), which gates and monitors access between a subject and a resource. The Policy Engine and Policy Administrator together form the Policy Decision Point.

2.3 Zero Trust Pillars

The Cybersecurity and Infrastructure Security Agency identifies five pillars: identity, devices, networks, applications and workloads, and data. Visibility and analytics, automation and orchestration, and governance support these pillars. Although intended for organizations, the pillars also provide a useful checklist for a personal AI assistant.

3. Threat Model

3.1 Assets

- Owner's files and documents.
- Messaging accounts accessible through the assistant.

- API keys and other secrets stored in the project folder.
- The PC, including running applications and system settings.
- Owner privacy, because the system uses a microphone and camera.

3.2 Threats and Failure Sources

T1 — Unauthorized speaker: A guest or family member gives commands near the PC.

T2 — Misrecognized speech: A non-malicious speech-recognition error triggers an unintended action.

T3 — Indirect prompt injection: Hostile instructions are embedded in files, webpages, or search results read by the assistant.

T4 — Model error: The model produces an incorrect tool argument, such as a wrong file path.

T5 — Generated code risk: The assistant writes a program that the owner may execute.

T6 — Credential exposure: An API key is leaked or misused.

3.3 Scope and Assumptions

The host operating system is assumed to be patched and trusted, one owner is enrolled, and the owner is physically present while the assistant is active. Attacks on the host operating system, physical theft, and compromise of the model provider are outside the scope of the study.

4. System Overview of Guts AI

Guts AI is implemented in Python 3.11 using asynchronous tasks. Microphone audio in 16 kHz PCM is streamed to the Gemini Live API. The service returns 24 kHz speech audio, transcripts, and potential function calls. The runtime executes declared functions and returns their results to the model.

4.1 Tool Capabilities

The prototype declares 51 functions covering system status and window information, file and folder operations, application and website launching, WhatsApp messaging, code writing and execution, web research, video editing, Cisco Packet Tracer lab creation, window management, and hardware controls such as volume and screenshots. Packet Tracer topologies are generated after a checker validates them.

4.2 Local Components and Privacy

An offline wake-word listener operates while the assistant is asleep, and no microphone audio is sent to the cloud during that state. Face verification runs locally on the PC. Only short text events, such as an unknown-person alert, are sent to the model. A watcher thread reports file changes and battery level using names and titles rather than file contents.

4.3 Credential Separation

Three API keys are used for separate purposes: real-time voice, everyday coding and research, and occasional heavy tasks. A fallback chain allows coding and research to continue with another provider when a limit is reached.

5. Zero Trust Design

The central design decision is to treat the language model as untrusted. The model may propose a declared tool call, but every call passes through gates implemented in ordinary code that the model cannot edit or approve. System prompts may encourage safe behavior, but they are not treated as an enforcement mechanism.

C1 — Continuous identity through face verification. The camera detects and matches the enrolled owner locally. Except for one read-only status call, tools require a recent owner match.

C2 — Spoken confirmation judged outside the model. Risky actions are held as pending. Python evaluates the owner's spoken yes/no response after the model has finished speaking. A 'no' cancels even if a 'yes' is also present; a yes requires a recent owner face match and expires after 45 seconds.

C3 — Allow-listed capabilities. Only declared functions are callable. Tools accept defined string parameters, and arbitrary shell execution is not exposed.

C4 — Path and file policy with scoped code execution. Sensitive system directories and protected project files cannot be modified. Generated code is stored in a new project folder, and only the most recently written file in that folder may be run. Risky-code keywords trigger owner review.

C5 — Separated API keys. Different keys are used for voice, coding/research, and heavy tasks to limit the impact of a leaked or exhausted credential.

C6 — Treat content as data and make actions recoverable. Text in files, webpages, and research results is not itself an instruction. File deletion uses the Recycle Bin, overwritten files are backed up first, and generated code does not overwrite earlier work.

C7 — Event logging. Tool calls and results, confirmation decisions, and face events are recorded. The logs support review but are not tamper-evident.

5.1 Face-Verification Decision Logic

The system uses YuNet face detection and SFace embeddings through OpenCV. For a detected face embedding f and enrolled gallery {gk}, the match score is the maximum cosine similarity: s(f) = maxk cos(f, gk). A score of at least 0.32 is treated as the owner; a score below 0.20 is treated as a stranger; values between these bounds are uncertain and do not trigger hostile behavior. A recent owner match is required for tool access.

5.2 Tool Risk Classes

Class	Required gate	Examples
Read-only / computation	Face gate	System status, list folder, research
Creates data in a new folder	Face gate	Generated code, lab plan
Changes/removes data or contacts others	Face gate + spoken yes	Write file, delete, WhatsApp, run program
Conditionally risky	Spoken yes when needed	Run code with risky calls, close browser
Not exposed	Unavailable	Arbitrary shell, secret-file read, system-folder write

6. Implementation Details

The system uses Python 3.11 and asyncio. PyAudio captures and plays audio. Face detection uses OpenCV's FaceDetectorYN with YuNet, while recognition uses FaceRecognizerSF with SFace. Enrollment captures up to 30 samples across five head poses—straight, left, right, up, and down—with only the owner in view. Matching runs approximately once per second in a background thread.

6.1 Main Parameters

Parameter	Value	Purpose
Match threshold	0.32	Owner if score meets threshold
Stranger bound	0.20	Below this score, classify as stranger
Adaptation score	0.55	Add sample to gallery
Gallery size	45 maximum	Limit adaptation drift
Owner grace for tools	90 seconds	Recent owner verification window
Owner grace for confirmation	15 seconds	Owner must be seen recently for yes
Confirmation expiry	45 seconds	Pending action timeout
Startup warm-up	12 seconds	Avoid stranger decisions during warm-up

6.2 Tool Dispatch

When the assistant is asleep, dispatch returns an ASLEEP result. For calls other than the read-only status call, the dispatcher checks whether the owner has been seen within the 90-second grace period. If a stranger is present, the request is denied; if the owner has not yet been verified, the system returns NOT VERIFIED. Otherwise, the declared tool is invoked. Pending risky actions are executed only after a valid spoken confirmation; a negative response cancels the action.

7. Evaluation and Results

7.1 Face Verification of the Owner

In test mode, the face module recorded one score per frame for 25 seconds. After enrollment, the owner sat in front of the webcam and 26 consecutive readings were collected. All 26 readings exceeded the match threshold of 0.32. The mean score was 0.644, standard deviation 0.066, and median 0.65. The minimum was 0.47 and maximum was 0.76; therefore, the lowest reading

exceeded the threshold by 0.15. A later full-assistant run reported a score of 0.75 and accepted the owner.

The evaluation is limited: it involved one person, one session, and one setting. No impostor readings were collected, so false-accept and false-reject rates are not reported.

7.2 Scenario Walkthroughs

Scenario	Controls	Evidence status
Owner sends a WhatsApp message by voice	C1, C2	Exercised in an early version
Generated calculator contains eval()	C4 scan and owner review	Exercised
Unknown person speaks to assistant	C1 (denial)	Design analysis
Hostile text hidden in a file	C6, C1, C2	Design analysis
Misheard delete command	C2, C6 (Recycle Bin)	Design analysis
One API key is leaked	C5	Design analysis

7.3 Cost of Security Friction

In the calculator scenario, the scanner flagged eval() in generated code. The program was harmless in its particular context, but the system required the owner to review it before execution. Reading the code and running it from an editor took seconds. This demonstrates the trade-off: stricter controls introduce friction, and a risk-based design must determine where that friction is justified.

8. Discussion and Limitations

The prototype demonstrates that Zero Trust principles can be applied to a personal AI assistant without modifying the underlying model. The enforcement gates reside in code around the model, so a hijacked model's proposed action faces the same checks as any other request.

Several limitations remain:

- There is no liveness detection; a printed or displayed photograph may pass the face gate.

- There is no speaker verification. If the owner and another person are both visible, the system cannot attribute speech to the correct speaker.
- Spoken confirmation depends on automatic speech transcripts, which may be inaccurate or influenced by nearby audio.
- A confused-deputy risk remains: the model may misuse an allowed tool within policy, such as selecting an incorrect but permitted file.
- The path policy uses a block list; a folder allow list would provide stronger restriction.
- Gallery adaptation could be abused by a look-alike whose score exceeds 0.55; adaptation should be capped and logged.
- Console logs can be modified after the fact and are not tamper-evident.
- The evaluation covers one user and one session, with no red-team testing.
- Malware running with the owner's privileges could bypass controls implemented in the same process; host compromise is out of scope.

8.1 Future Work

Future improvements include liveness detection, speaker verification, a signed append-only audit log, per-tool permission tokens with expiration, folder allow lists, anomaly detection (for example, unusual deletion activity), and broader testing with multiple impostors and a corpus of prompt-injection attempts. These studies could measure false-accept and false-reject rates and strengthen evidence for the proposed controls.

The NIST AI Risk Management Framework describes the functions Govern, Map, Measure, and Manage. A planned audit log and red-team exercises align with the Measure and Manage functions.

9. Conclusion

This report applied Zero Trust Architecture to Guts AI, a voice-controlled assistant with authority over a personal computer. Seven controls translate the principles of explicit verification, least privilege, assumed breach, and visibility into practical enforcement: a local face gate, spoken confirmation judged outside the model, allow-listed tools without shell access, path policies and scoped code execution, separated API keys, recoverable actions with content treated as data, and event logging. All 26 owner face readings in the reported test exceeded the 0.32 threshold, with a mean score of 0.644. The code scanner also stopped an unreviewed

generated program. Because the evaluation is small and several scenarios are only design analyses, the results should be considered an initial case study and baseline for stronger testing. The central lesson is that trust should be placed in independently enforced checks, not solely in a model's willingness to obey.

Acknowledgment

The author thanks the open-source communities behind Python and OpenCV, whose tools made this prototype possible.

References

[1] J. Kindervag, "No more chewy centers: Introducing the zero trust model of information security," Forrester Research, Cambridge, MA, USA, Sep. 2010.

[2] R. Ward and B. Beyer, "BeyondCorp: A new approach to enterprise security," ;login:, vol. 39, no. 6, pp. 6–11, Dec. 2014.

[3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," National Institute of Standards and Technology, NIST SP 800-207, Aug. 2020, doi: 10.6028/NIST.SP.800-207.

[4] OWASP Foundation, "OWASP top 10 for large language model applications," version 1.1, 2023.

[5] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, arXiv:2302.12173.

[6] Google, "Live API," Gemini API documentation, Google AI for Developers. [Online]. Available: https://ai.google.dev/gemini-api/docs/live (accessed Oct. 2026).

[7] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," Proc. IEEE, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.

[8] Cybersecurity and Infrastructure Security Agency, "Zero trust maturity model," version 2.0, Apr. 2023.

[9] N. N. Kyaw, "Analysis and simulation of hyper text transfer protocol at the application layer of the Internet," Int. J. Sci. Res. Publ., vol. 9, no. 1, pp. 78–84, Jan. 2019, doi: 10.29322/IJSRP.9.01.2019.p8512.

[10] W. Wu, H. Peng, and S. Yu, "YuNet: A tiny millisecond-level face detector," Mach. Intell. Res., vol. 20, pp. 656–665, 2023.

[11] Y. Zhong, W. Deng, J. Hu, D. Zhao, X. Li, and D. Wen, “SFace: Sigmoid-constrained hypersphere loss for robust face recognition,” IEEE Trans. Image Process., vol. 30, pp. 2587–2598, 2021.

[12] G. Bradski, “The OpenCV library,” Dr. Dobb’s Journal of Software Tools, 2000.

[13] ISO/IEC 30107-3:2017, “Information technology—Biometric presentation attack detection—Part 3: Testing and reporting,” 2017.

[14] National Institute of Standards and Technology, “Artificial intelligence risk management framework (AI RMF 1.0),” NIST AI 100-1, Jan. 2023.